

Инструкция оператора Blank VPN Bot Installer

Эта инструкция описывает установку пустого VPN-бота без бренда: бот, RemnaWave-панель, страница подписки, кабинет, базовые алиасы, нейтральные стартовые баннеры, Telegram Stars по умолчанию и команды для добавления серверов, платежей и баннеров.

1. Что подготовить до запуска

Нужен свежий сервер Ubuntu с root-доступом, публичный IPv4 и открытыми портами 80/tcp, 443/tcp, 443/udp.

Нужен домен. Можно использовать Cloudflare или ручную настройку DNS. Если используется Cloudflare, поддомен подписки `sub.example.com` обязательно должен быть DNS-only, без проксирования через Cloudflare. Остальные публичные поддомены можно проксировать, но по умолчанию безопаснее оставить DNS-only до первой проверки.

Нужен Telegram-бот, созданный через BotFather:

- token бота;
- username бота без @;
- Telegram ID админа или нескольких админов через запятую.

Нужен Telegram поддержки и режим поддержки:

- `tickets + contact` - в боте есть тикеты и контакт поддержки;
- `tickets only` - только тикеты;
- `contact only` - только контакт поддержки.

По умолчанию отдельный репозиторий бота не нужен: установщик берет код из `bot-source/` внутри своего репозитория. Внешний Git-репозиторий можно выбрать только как дополнительный режим.

2. Запуск установки

На свежем сервере:

```
curl -fsSL https://raw.githubusercontent.com/cinemabit55/blank-vpn-bot-installer/main/scripts/install_blank_vpn_bot.sh
-o /tmp/install_blank_vpn_bot.sh
sudo bash /tmp/install_blank_vpn_bot.sh
```

Если репозиторий установщика private, нужен GitHub token с доступом на чтение:

```
export INSTALLER_GITHUB_TOKEN=github_pat_or_classic_token
curl -fsSL -H "Authorization: Bearer $INSTALLER_GITHUB_TOKEN" \
https://raw.githubusercontent.com/cinemabit55/blank-vpn-bot-installer/main/scripts/install_blank_vpn_bot.sh
-o /tmp/install_blank_vpn_bot.sh
sudo INSTALLER_GITHUB_TOKEN="$INSTALLER_GITHUB_TOKEN" bash /tmp/install_blank_vpn_bot.sh
```

Пока GitHub-репозиторий установщика не опубликован, можно запускать из локального checkout:

```
sudo bash scripts/install_blank_vpn_bot.sh
```

Скрипт показывает статус каждого этапа: пакеты, Docker, DNS, подготовка bundled-кода бота, генерация и проверка конфигов, запуск контейнеров, установка алиасов и итоговый summary.

Повторный запуск не удаляет runtime-файлы в директориях установки: data, logs, uploads и другие появившиеся файлы сохраняются. Если уже есть /opt/blank-vpn-bot-installer/answers.last.json, установщик предложит переиспользовать прошлые ответы, а в non-interactive режиме сделает это автоматически. Если отвечать заново, старые сгенерированные секреты все равно сохраняются, пока их явно не передали через answers-файл.

Полезные флаги для повторного запуска:

--reuse-answers	использовать /opt/blank-vpn-bot-installer/answers.last.json без вопроса
--no-reuse-answers	пройти вопросы заново, сохранив старые сгенерированные секреты
--skip-validation	пропустить docker compose config и caddy validate

3. Что спрашивает установщик

Установщик в обычном режиме использует bundled bot-source/ из репозитория установщика и не спрашивает источник кода. Для advanced-сценария можно передать внешний репозиторий флагами:

```
--source-mode git --source-repo REPO_URL --source-ref BRANCH_OR_TAG
```

Дальше установщик попросит:

- директорию установки, обычно /opt;
- название проекта, которое попадет в кабинет;
- IPv4 сервера;
- корневой домен;
- домены панели, подписки, кабинета и API;
- email для Let's Encrypt;
- режим DNS: manual или Cloudflare;
- Cloudflare API token, если выбран Cloudflare;
- token Telegram-бота;
- username Telegram-бота;
- Telegram ID админов;
- Telegram поддержки;
- режим поддержки;
- курс Telegram Stars;
- RemnaWave admin username;
- RemnaWave admin password, либо пустое значение для автогенерации;
- RemnaWave API token, если он уже есть. Если оставить пустым, установщик создаст token сам.

В конце установщик выводит ссылки, логины, важные токены и список команд. Копия summary сохраняется в:

```
/opt/blank-vpn-bot-installer/install-summary.txt
```

Файл доступен только root.

4. DNS

Нужно четыре A-записи на IPv4 сервера:

panel.example.com	A	SERVER_IP
sub.example.com	A	SERVER_IP
cabinet.example.com	A	SERVER_IP
api.example.com	A	SERVER_IP

Правило для Cloudflare: sub.example.com всегда DNS-only. Его нельзя проксировать, иначе HAP/Proxy-подписка и некоторые проверки могут работать нестабильно.

5. RemnaWave admin и API token

Обычный сценарий полностью автоматический:

1. Установщик запускает RemnaWave. 2. Ждет локальный API `http://127.0.0.1:3000/api/auth/status`.
3. Если регистрация открыта, создает admin-пользователя. 4. Логинится и получает admin JWT. 5. Создает долгоживущий API token через `/api/tokens`. 6. Записывает token в `/opt/bedolaga/.env` и `/opt/remnawave/.env.subscription`. 7. Проверяет сгенерированные Docker Compose файлы и Caddyfile.
8. После этого запускает бот, кабинет, страницу подписки и Caddy. 9. При первом старте бота создает тарифы Базовый, Темные списки, Триал, если их еще нет.

RemnaWave admin login/password и API token будут в финальном файле:

```
/opt/blank-vpn-bot-installer/install-summary.txt
```

Файл доступен только root.

6. Команды после установки

Основные команды:

add_payment	добавить платежный метод
add_banner	интерактивно заменить баннер
set_banner	заменить баннер одной командой
list_banners	показать установленные баннеры
reset_banner	убрать баннер после backup
bot_commands	показать bot-команды
add_direct	добавить прямое подключение
add_cascade	добавить каскадное подключение
add_inbound	добавить RU edge к иностранной ноде
add_routes	обновить RU direct routing
delete_node	удалить подключение
change_sni	сменить SNI/маскировку
node_commands	показать node-команды

Скрипты добавления нод берутся из актуального репозитория бота. В них должны быть заложены последние настройки: TCP REALITY по умолчанию, XHTTP только как совместимый старый режим, server keepalive, Firefox/Safari SNI по выбранному сценарию, HAP-заголовки, фрагментация, общий no-limit для TCP, маршрутизация RU direct, Whoosh direct и UDP direct/WARP-логика из текущей версии бота.

На чистой установке бот сам создает тарифы Базовый, Темные списки, Триал. Алиасы добавления нод используют эти имена, поэтому вручную создавать тарифы перед первой нодой не нужно. Если владелец уже изменил тарифы в кабинете, startup bootstrap их не перезаписывает.

7. Добавление платежки

По умолчанию включен только Telegram Stars.

Чтобы добавить платежку:

```
add_payment
```

Команда спросит платежный метод, название кнопки в боте, ключи/API/ID магазина и дополнительные URL. После этого она:

- делает backup /opt/bedolaga/.env;
- включает нужные env-переменные;
- пересоздает контейнер бота через `docker compose up -d --force-recreate bot`;
- включает метод в таблице `payment_method_configs`, если база уже готова.

Список поддерживаемых методов:

```
telegram_stars, tribute, cryptobot, heleket, yookassa, mulenpay, pal24,  
platega, wata, cloudpayments, freekassa, kassa_ai, riopay, severpay,  
paupear, rollypay, overpay, aurapay
```

Посмотреть список без изменений:

```
add_payment --list
```

8. Баннеры

Во время установки создается нейтральный banner pack для основных экранов бота. Он нужен только как стартовая заглушка: оператор может заменить любой слот после установки.

Загрузить баннер интерактивно:

```
add_banner
```

Заменить конкретный слот:

```
set_banner main_menu ru /root/banner.png  
set_banner profile all /root/profile.webp
```

Проверить состояние:

```
list_banners
```

Слоты: `main_menu`, `profile`, `referral`, `support`, `download`, `about`, `resources`, `welcome`.

Языки: `ru`, `en`, `fallback`, `all`.

9. Добавление серверов

Для нового пустого бота сначала нужно поднять сам control-plane, затем добавлять ноды алиасами.

Прямое иностранное подключение:

```
add_direct
```

Каскад:

```
add_cascade
```

Добавить RU edge к существующему иностранному выходу:

```
add_inbound
```

Обновить маршруты:

```
add_routes
```

Удалить ноду:

```
delete_node
```

Сменить SNI/маскировку:

```
change_sni
```

10. Проверка после установки

Проверь:

- <https://panel.example.com> открывает RemnaWave;
- <https://sub.example.com/connect> открывает страницу подписки;
- <https://cabinet.example.com> открывает кабинет;
- бот отвечает в Telegram;
- `docker compose ps` в `/opt/bedolaga`, `/opt/remnawave`, `/opt/caddy-remnawave` без постоянных рестартов.

Логи:

```
cd /opt/bedolaga && docker compose logs -f bot
cd /opt/remnawave && docker compose logs -f
cd /opt/caddy-remnawave && docker compose logs -f
```

11. Типовые проблемы

Если панель или подписка не открываются, сначала проверь DNS и Cloudflare proxy mode. sub должен быть DNS-only.

Если бот не видит RemnaWave, проверь REMNAWAVE_API_KEY в `/opt/bedolaga/.env`.

Если страница подписки не работает, проверь REMNAWAVE_API_TOKEN в `/opt/remnawave/.env.subscription`.

Если платежка не появилась после `add_payment`, открой кабинет и проверь раздел платежных методов. Env уже сохранен, но видимость можно включить вручную.

Если после правки `.env` бот не подхватил изменения, используй:

```
cd /opt/bedolaga && docker compose up -d --force-recreate bot
```